

Relatório de Implementação — Etapa 2

Sistema de Gestão Hospitalar Dra. Yuska Maritan Brito

Beatriz Pessoa e Emyle Santos · Banco de Dados

1. Triggers ou stored procedures: qual foi o critério

A pergunta não é qual tecnologia é melhor, e sim **de quem é a responsabilidade pela regra**. Adotamos uma ordem de preferência: *constraint declarativa* → *trigger* → *procedure* → *código da aplicação*, descendo um degrau só quando o anterior não dá conta.

Constraint, quando a regra cabe numa. A `UNIQUE(id_unidade, dia_semana, turno, id_residente)` da Etapa 1 já impede o mesmo residente duas vezes no mesmo plantão. É garantida por índice, sai de graça e é segura sob concorrência.

Trigger, para invariantes que valem para qualquer escrita. Três casos:

- `trg_check_sobreposicao_escala` cobre o que a constraint **não** consegue expressar: o mesmo residente escalado no mesmo dia/turno em **unidades diferentes**. É uma comparação entre linhas, não uma unicidade de coluna, e por isso precisa de trigger.
- `trg_audita_atendimento` precisa registrar toda alteração em `atendimento`, inclusive as feitas direto no `psql`, fora da aplicação. Uma auditoria que pode ser burlada não é auditoria. Esse é o argumento decisivo a favor do trigger aqui.
- `trg_atualiza_media_procedimentos` mantém um dado derivado (`media_tempo_procedimento`) que precisa acompanhar cada inserção. Deixar isso na aplicação significaria esquecer de atualizar em algum caminho de código.

Procedure, para operações compostas que alguém invoca de propósito. Um trigger não recebe parâmetros nem é chamado sob demanda; quando a operação tem entrada e é acionada por decisão do usuário, o lugar é a procedure:

- `sp_registrar_atendimento_completo` recebe o atendimento e uma lista JSON de procedimentos e grava tudo numa transação. Não há bloco `EXCEPTION`: um `CALL` já roda como transação implícita, então qualquer erro reverte também o atendimento inserido antes do laço. Um `EXCEPTION` que engolisse o erro faria o oposto do pedido.
- `sp_reajustar_escala` move em lote os plantões de um residente. Ela valida **antes** do `UPDATE`: se qualquer escala colidir no destino, nenhuma é movida. As validações são explícitas em vez de delegadas ao `CHECK` da tabela, para a mensagem dizer o que houve em vez de citar uma constraint.
- `sp_calcular_tempo_medio_espera` é um relatório sob demanda. Como procedures no PostgreSQL não têm `RETURNS TABLE`, o resultado sai por `REFCURSOR`, que só vive dentro da transação — por isso a aplicação faz o `CALL` e o `FETCH` na mesma sessão, antes do commit.

Views ficaram com as consultas de leitura reaproveitáveis (`vw_pacientes_internados`, `vw_residentes_sem_supervisor`, `vw_estatisticas_atendimentos_mensal`): não têm efeito colateral nem parâmetro, e podem ser consultadas como se fossem tabelas.

2. Escolha da ORM: SQLAlchemy

Escolhemos **SQLAlchemy** por um motivo concreto além da recomendação do enunciado: ele permite descer ao SQL sem sair da sessão. Views e procedures não são entidades mapeáveis, e com `session.execute()` conseguimos consumi-las **dentro da mesma transação** do restante da operação. O Django ORM obrigaria a reescrever a aplicação inteira em Django por causa de uma camada de dados.

O problema da especialização. O schema da Etapa 1 modela `PESSOA` → `PACIENTE` / `PROFISSIONAL` → `PRECEPTOR` / `RESIDENTE` com FK-como-PK, mas **sem coluna discriminadora**. A herança de tabelas do SQLAlchemy precisa dessa coluna para saber qual subclasse instanciar. Adicioná-la mudaria um schema já entregue e avaliado, então mapeamos a especialização por **composição**: cada subtipo tem um relacionamento 1:1 com o supertipo (`Paciente.pessoa`), e `Residente.pessoa` é uma `@property` que atravessa `profissional`. Perdemos consultas polimórficas — que o sistema não usa — e mantivemos o schema fiel.

3. Lazy vs. eager loading: decidido por tela e medido

A escolha foi feita relacionamento a relacionamento, olhando o que cada tela exhibe:

- `joined` nos 1:1 sempre exibidos juntos (`Paciente.pessoa`, `Escala.unidade`): com lazy, toda listagem faria uma query extra por linha.
- `selectin` na coleção `Paciente.alergias`: com `joined`, o JOIN multiplicaria as linhas do paciente (uma por alergia); `selectin` resolve em uma segunda query só.
- `lazy` nos caminhos que nenhuma tela percorre (`Alergia.pacientes`, `Unidade.escalas`).

Medindo os `SELECT` emitidos: listar 6 pacientes com alergias custa **2 queries** (seriam 7 com lazy), e listar 10 escalas com unidade, residente e preceptor encadeados custa **1 query** (seriam mais de 30).

Essa escolha teve uma consequência que só apareceu depois: `Residente.profissional` é eager `joined`, o que transforma a consulta em `LEFT OUTER JOIN` — e o PostgreSQL **recusa** `FOR UPDATE` sobre o lado anulável de um outer join. Na demo de lock pessimista foi preciso desligar o eager com `lazyload()` naquela query específica.

4. Concorrência: lock pessimista e otimista

O enunciado pedia um dos dois; implementamos os dois, cada um com um cenário de contraste que mostra o que acontece **sem** o mecanismo.

Pessimista (`orm/concorrencencia_pessimista.py`). A escala disputada ainda não existe — é ela que está sendo inserida —, então não há linha de escala para travar. Travamos a linha do **residente**, que é o recurso realmente disputado, com `SELECT ... FOR UPDATE`. Sem o lock, as duas transações verificam "não existe", as duas inserem, e uma estoura na constraint. Com o lock, a segunda espera (~156 ms), relê, vê que a escala já existe e desiste sem erro: a **UNIQUE** nunca chega a ser acionada. Isso depende do PostgreSQL rodar em **READ COMMITTED**, em que cada comando tira um snapshot novo; em **REPEATABLE READ** a transação que esperou manteria o snapshot antigo, não veria a escala recém-criada e cairia na constraint mesmo com o lock.

Otimista (`orm/concorrencencia_otimista.py`). Uma coluna `version_id` em `escala`, mapeada com o `version_id_col` do SQLAlchemy, que adiciona `WHERE version_id = ?` a cada `UPDATE`. O contraste aqui é ainda mais didático: **sem** versionamento as duas transações commitam com sucesso, ninguém recebe erro, e mesmo assim uma das escritas é sobrescrita em silêncio — um *lost update*. Com versionamento, a segunda a commitar recebe `StaleDataError` e desfaz tudo.

Quando usar cada um. O pessimista serializa de fato e evita retrabalho, mas faz a segunda transação esperar e segura lock no banco — cabe em transações curtas e disputa provável, como escalar um residente. O otimista não bloqueia ninguém, mas transfere o custo para o cliente, que precisa reler e tentar de novo — cabe quando o conflito é raro. O critério prático é o custo relativo entre esperar e refazer.

5. Mudanças de schema exigidas pela Etapa 2

A Etapa 2 pediu recursos que o schema da Etapa 1 não comportava: duas tabelas novas (`internacao`, `auditoria_atendimento`) e quatro colunas (`procedimento.media_tempo_procedimento`, `procedimento_realizado.hora_inicio`, `atendimento.id_unidade`, `escala.version_id`).

`atendimento.id_unidade` merece registro: o enunciado pede o tempo médio de espera "para cada unidade" e estatísticas "por mês e por unidade", mas o schema que ele mesmo define na Etapa 1 não liga atendimento a unidade. Sem essa FK, as duas agregações não teriam por onde agrupar.

Duas decisões de projeto nessas tabelas: `auditoria_atendimento` **não tem FK** para `atendimento` — com FK (mesmo `ON DELETE CASCADE`), apagar um atendimento apagaria o próprio registro do DELETE, e o rastro se autodestruiria. E `vw_pacientes_internados` usa `ROW_NUMBER()` para olhar apenas a internação **mais recente** de cada paciente: uma consulta ingênua por "existe internação em aberto" traria também quem tem uma internação antiga que ninguém fechou.

Todos os scripts são reexecutáveis (`IF NOT EXISTS`, `CREATE OR REPLACE`, `INSERT` condicional), para que rodar a instalação duas vezes não duplique dados nem quebre.

6. Repositório e interface

O histórico separa as etapas: 28 commits da Etapa 1 e 14 da Etapa 2, a partir de [9a08c85](#). Toda funcionalidade da Etapa 2 é alcançável pela interface — internações, auditoria com destaque dos campos alterados, as três views, as procedures e as consultas avançadas —, e cada bloco indica de qual recurso do banco veio o dado. As duas demos de concorrência rodam por linha de comando, por precisarem de duas transações simultâneas com log da ordem de execução.